Reviewer Pack — Commutator-Length Defect in S_5 Predicts Width-5 BP Size of …

Entry bbfdc40acf2e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 21:48:03 UTC

Commutator-Length Defect in S_5 Predicts Width-5 BP Size of Boolean Formulas

Entry ID: `bbfdc40acf2e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 21:48:03 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial group theory of S_5: commutator length cl(g) (minimum k with g = [a_1,b_1]...[a_k,b_k]) and its slack against the trivial bound, computed by exact BFS in the Cayley graph of S_5 under conjugation moves **Field B** (complexity object): Width-5 permutation branching program (BP) length L_5(F) representing a Boolean formula F via Barrington's construction; equivalently a quantitative formula-size proxy in NC^1

Statement:

> For every Boolean formula F over n<=20 variables and depth d<=6, let phi(F) be the unique 5-cycle product produced by Barrington's standard inductive construction on F, and define the commutator-length defect D(F) = 4*depth(F) - cl(phi(F)) where cl is the S_5 commutator length of phi(F) when 1=accept, 1=reject distinguishes by 5-cycles. Then the optimal width-5 BP length satisfies L_5(F) >= 2^{D(F)} (2depth(F) + 1), i.e., a positive commutator-length defect forces an exponential-in-defect blow-up over the trivial 4^depth Barrington bound. Concretely: every formula F with D(F) >= 1 admits no width-5 BP of length < 2*(2*depth(F)+1).

Rationale (proposer's reasoning):

> Barrington's theorem encodes formula evaluation by iterated commutators of 5-cycles, but the only structural invariant ever extracted from the resulting permutation is its cycle type; commutator length in S_5 is a finer, rarely-used algebraic statistic that measures how 'cheaply' phi(F) can be re-expressed as commutators, potentially detecting hidden algebraic redundancy that a shorter BP would have to exploit. If a short BP exists, the realized permutation must be writable with few commutators, so a large defect should obstruct compression; this connects geometric group theory's stable commutator length tradition (Calegari, Bavard) to NC^1 lower bounds in a way that has, to our knowledge, not been pursued.

Taxonomy category: `BARRINGTON_ALG` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3945f64bcacb1355`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across ~300 random Boolean formulas (depth 2-6, n=4-8) tested on >=5 RNG seeds, the conjecture is SUPPORTED iff zero instances with defect D(F)>=1 admit a width-5 BP of length < 2*(2*depth(F)+1), AND the median ratio L_5(F)/(2^{D(F)}*(2*depth(F)+1)) >= 1 over the seed-aggregated sample.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Barrington width-5 permutation branching program commutator length S_5 - NC1 formula size lower bound symmetric group commutator width branching program - Cayley graph S_5 commutator length Boolean formula branching program size`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import json

def generate_random_formula(n, d):
    if n == 0 or d == 0:
        return "NOT"
    elif d == 1:
        return random.choice(["AND", "OR"])
    else:
        subformulas = [generate_random_formula(random.randint(0, n-1), random.randint(1, d-1)) for
        operator = random.choice(["AND", "OR"])
        return f"({subformulas[0]} {operator} {subformulas[1]})"

def multiply_permutations(p, q):
    n = len(p)
    result = [0] * n
    for i in range(n):
        result[i] = p[q[i]]
    return result
```

```
def identity_permutation(n):
    return list(range(n))

def commutator_length(permutation):
    n = len(permutation)
    queue = [(identity_permutation(n), 0)]
    visited = set()
    while queue:
        current, length = queue.pop(0)
        if current == permutation:
            return length
        for a in range(n):
            for b in range(a+1, n):
                commutator = multiply_permutations(multiply_permutations([a], [b]), multiply_permu
                next_permutation = multiply_permutations(current, commutator)
                if tuple(next_permutation) not in visited:
                    visited.add(tuple(next_permutation))
                    queue.append((next_permutation, length + 1))
    return float('inf')

def find_min_bp_length(permutation):
    n = len(permutation)
    instructions = []
    current = identity_permutation(n)
    for i in range(2**n):
        if current == permutation:
            break
        for a in range(n):
            for b in range(a+1, n):
                commutator = multiply_permutations([a], [b])
                next_permutation = multiply_permutations(current, commutator)
                if next_permutation == permutation:
                    instructions.append((a, b))
                    current = next_permutation
                    break
            else:
                continue
            break
    return len(instructions)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 5, 6, 7, 8]
    d_values = [2, 3, 4, 5, 6]
    results = []

    for n in n_values:
        for d in d_values:
            F = generate_random_formula(n, d)
            phi_F = Barrington_construction(F)  # Assume this function is defined elsewhere
            cl_phi_F = commutator_length(phi_F)
            D_F = 4 * d - cl_phi_F
            L_5_F = find_min_bp_length(phi_F)

            results.append({
                "n": n,
```

```
                "d": d,
                "F": F,
                "phi_F": phi_F,
                "cl_phi_F": cl_phi_F,
                "D_F": D_F,
                "L_5_F": L_5_F
            })

    conjecture_holds = all(D_F < 1 or L_5_F >= 2 * (2 * d + 1) for result in results)
    counterexample = "" if conjecture_holds else "D(F)>=1 and L_5(F)<2*(2*d+1)"

    return {
        "metric_name": "L_5/F",
        "metric_value": sum(result["L_5_F"] / (2**(result["D_F"]) * (2*result["d"] + 1)) for resul
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) if arg.isdigit() else int.from_bytes(arg.encode(), 'big') for arg in sys.arg
    if not seeds:
        seeds = [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_L_5_over_F = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_L_5_over_F} std=0.0 support_fraction={support_fractio
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"D(F)>=1 and L_5(F)<2*(2*d+1)\" first_failing_se
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa494294.py", line 117, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa494294.py", line 84, in run_trial
    phi_F = Barrington_construction(F)  # Assume this function is defined elsewhere
            ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
NameError: name 'Barrington_construction' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a NameError because Barrington_construction was never defined, so no data was produced and the pre-registered support condition cannot be evaluated. | next: Implement Barrington's standard inductive construction (mapping AND/OR/NOT gates to 5-cycle products in S_5) and re-run the trial to actually compute D(F) and L_5(F).

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	23073
2	preregistration	claude_max	opus	0	0	5884
3	novelty	claude_max	opus	0	0	2853
4	novelty	claude_max	opus	0	0	6558
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15215
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13128
7	judge	claude_max	opus	0	0	5183

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 71894 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/bbfdc40acf2e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bbfdc40acf2e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bbfdc40acf2e.tar.gz` (if generated)